Lessons Learned – Shail Shah

CS6650 Final Project

April 2026

What I worked on

My primary areas were the `waiting-room` service (Redis ZADD-based queue, two scoring strategies, token-bucket admission controller), the Terraform infrastructure (8 modules: network, ALB, ECS, ECR, Redis, SQS, DynamoDB, logging), and the CI/CD deploy scripts for all three services.

What went well

The infrastructure stayed understandable even as the project grew. Breaking Terraform into eight small modules kept the AWS side legible: networking, ECS, Redis, SQS, DynamoDB, ECR, and logging could evolve without every change turning into one giant plan diff. That mattered in the last week when service code was still changing and we needed to be able to reason about whether a problem was infrastructure, application, or experiment harness.

The admission-ticker pattern itself was the right abstraction. Once the queue positions and token issuance were separated, the waiting-room became a real control surface instead of just a list of who clicked first. Even before the downstream enforcement bug was fixed, the waiting-room code had the right mental model: fairness at the front, admission rate as a tunable knob, and Redis as the single serialized state owner.

The deploy story was consistent across services. Having one ECS/Fargate deployment pattern for all three services made the team faster. The scripts were not glamorous work, but they kept us from debugging three different release processes on top of the actual distributed-systems problems.

What went wrong

Two concurrency bugs shipped to main in the waiting-room service. The `timestamp_incr` strategy was silently a no-op due to float64 precision loss, and the separate-round-trip `ZADD`/`ZRANK` pair had a rank-displacement race under concurrent inserts with lex-smaller member IDs. Both passed code review, both passed the existing unit tests, and both were caught only when Experiment 1's sweep produced collision rates that were identical between the two strategies. Fix was a single-file change (atomic Lua script for INCR + ZADD + ZRANK) but the time to diagnose was a full afternoon.

What made these bugs hard to see is that the code *looked* reasonable. `timestamp + tiny fractional tiebreaker` sounds correct at a glance, and `ZADD` followed by `ZRANK` sounds like a harmless two-step read-back. The review process missed the fact that correctness here depended on numeric precision and on the

exact atomicity boundary. Both bugs lived in the gap between "the code reads well" and "the distributed behavior is actually invariant-preserving."

Admission token issuance without enforcement. The waiting-room issued admission tokens from week 2. The flash-sale-api never checked them. Without the gate, Experiment 3's admission-rate tuning would have been meaningless – the "rate limiter" wasn't actually rate-limiting anything downstream. We caught this while reading the flash-sale-api source for another reason.

How the course concepts applied

- Single-threaded-counter coordination. Redis's single-threaded execution gave us a free coordination primitive that underpinned both the queue (via `INCR` as score) and the inventory (via `DECRBY`). The correct pattern in both cases was "do everything on the server atomically," not "read, decide on the client, write." The course spent several lectures on this exact contrast.
- Sorted-set semantics. The `ZRANK`/`ZADD` race we hit is a specific instance of the broader lesson that "rank" is a view-of-the-world, not a property of a member. Two callers observing the same member's rank at different times can see different values, and two different members can have the same rank at different times. The fix was to freeze the view: an atomic `ZADD + ZRANK` on the server, not two round-trips.
- Token-bucket rate limiting. The admission ticker is a classic token bucket. What we did not do correctly was wire it to the downstream consumer – a reminder that a correct rate limiter on its own is useless if nothing is reading its output.

What I would do differently

1. Run an experiment against my service on the day I finish it, with even a trivial load, to catch bugs that unit tests can't see.
2. Add end-to-end contract tests between services – if we had required a waiting-room token in a flash-sale-api integration test, the missing gate would have been obvious in week 2.
3. Treat "rank" and "position" as distributed invariants, not UI values. If a number is meant to represent fairness, it needs an explicit atomicity guarantee in the implementation and a stress test in CI.